

Vektorok C++-ban: az `std::vector`

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ezt a jegyzetet önállóan, a saját tempódban dolgozd fel: olvass, próbáld ki, aztán oldd meg a feladatokat. Háromféle rész váltja egymást:

- **Próbáld ki** – gépeld be a kódot, és futtasd. Gépeld, ne másold: a kezed is tanul.
- **Tippelj** – előbb írd le a füzetedbe, szerinted mit ír ki a program, és csak utána futtasd. Ha tévedtél, annál jobb: most tanultál valamit.
- **Feladat** – ezt már te írod meg. A feladatok kis lépésekben nehezednek, és egymásra épülnek. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Néhány tudnivaló, mielőtt belevágsz:

- Minden programba kell az `#include <vector>` sor az `#include <iostream>` mellé.
- A rövid kódrészleteket a `main` belsejébe írd.
- A kiírt szövegekben nincs ékezet, mert a konzol nem mindig jeleníti meg jól.
- Ha elakadsz: olvasd el a hibaüzenetet, nézd vissza a mintát, aztán kérdezz bátran.
- Nem baj, ha nem érsz a végére. Az számít, hogy amit megírtál, azt érted.

1. Miért kell a vektor?

A vektor sok azonos típusú adatot tárol egyetlen név alatt. Hogy miért jó ez, azt úgy érted meg a legjobban, ha előbb megpróbálsz nélküle.

Próbáld ki

Ebben a programban öt jegyedet tároljuk, öt külön változóban:

```
#include <iostream>

int main() {
    int jegy1 = 5;
    int jegy2 = 3;
    int jegy3 = 4;
    int jegy4 = 5;
    int jegy5 = 2;

    std::cout << jegy1 << " " << jegy2 << " " << jegy3 << " "
              << jegy4 << " " << jegy5 << std::endl;
    return 0;
}
```

Feladatok

1.1. feladat ★☆☆ - Öt változó

Gépeld be a programot, és futtasd.

1.2. feladat ★☆☆ - Még két jegy

Kaptál még két jegyet: egy 4-est és egy 5-öst. Bővítsd a programot úgy, hogy mind a hét jegyet kiírja. Számold meg: hány helyen kellett belenyúlnod a kódba?

1.3. feladat ★☆☆ - Harminc jegy

Ezt csak gondold végig, nem kell megírni. Év végére 30 jegyed lesz. Hogyan nézne ki a program? És ha előre nem is tudod, hány jegyed lesz?

Mit vettél észre?

Külön változókkal minden új adat új sort és új nevet kér. A számítógép pedig nem tudja, hogy a `jegy1` és a `jegy2` összetartozik: nem mondhatod neki, hogy „menj végig az összes jegyen”.

A vektor ezt oldja meg. Képzeld el úgy, mint egy sor számozott fiókot: a sornak egy neve van, a fiókoknak pedig sorszámuk.

Sorszám (index)	0	1	2	3	4
Érték	5	3	4	5	2

Figyeld meg a felső sort: a számozás nem 1-gyel kezdődik. Erre mindjárt visszatérünk.

2. Az első vektorunk

Egy vektort egyetlen sorral hozol létre, az elemeit pedig a sorszámukkal éred el.

Próbáld ki

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> jegyek = {5, 3, 4, 5, 2};

    std::cout << jegyek[0] << std::endl;
    return 0;
}
```

Nézzük meg az új sort darabonként:

- `std::vector<int>` – egy vektor, amelyben egész számok (`int`) vannak.
- `jegyek` – a vektor neve. Ezt te választod, mint bármely változónál.
- `{5, 3, 4, 5, 2}` – a kezdő elemek, kapcsos zárójelben, vesszővel elválasztva.
- `jegyek[0]` – a vektor egyik eleme. A szögletes zárójelben álló szám az **index**.

Tippelj

Mit ír ki a fenti program? Írd le a tipped, aztán futtasd.

Most írd át a `jegyek[0]`-t `jegyek[1]`-re, majd `jegyek[4]`-re. Mindkétszer tippelj előbb.

Válaszolj a füzetedben:

- Mi az első elem indexe?
- Mi az utolsó elem indexe, ha öt elem van?
- Mi lenne az utolsó elem indexe, ha száz elem lenne?

Kísérlet - mi van a vektor végén túl?

Írasd ki a `jegyek[5]` értékét. Aztán a `jegyek[100]` értékét is. Mit tapasztalsz? Futtasd többször is.

Figyelem! A C++ nem ellenőrzi, hogy létező elemet kérsz-e. Nem szól, csak furcsa számot ad, vagy leáll a program. Ez a **túlindexelés**: az index helyességére neked kell figyelned.

Feladatok

2.1. feladat ★☆☆ - Lépésszámláló

Hozz létre egy **lepesek** nevű vektort négy nap lépésszámával: 8200, 10450, 6300, 12000. Írd ki az első és az utolsó elemét.

2.2. feladat ★☆☆ - Egy sorban

Írd ki mind a négy elemet egy sorba, szóközzel elválasztva. (Ciklus most még nem kell.)

2.3. feladat ★☆☆ - Két nap összege

Írd ki az első két nap lépésszámának összegét. A program számoljon, ne te.

2.4. feladat ★☆☆ - Tizedes törtek

A vektorban nem csak egész számok lehetnek. Próbáld ki ezt a sort, és írd ki a középső elemet:

```
std::vector<double> arak = {299.9, 1250.5, 89.0};
```

Mi változott a létrehozásban az előzőhöz képest?

2.5. feladat ★☆☆ - Kedvenc számok

Hozz létre egy saját vektort a kedvenc öt számodból, és írd ki őket fordított sorrendben.

3. Elemek módosítása

A vektor egy eleme pontosan úgy viselkedik, mint egy sima változó: kaphat új értéket, lehet vele számolni, be lehet olvasni.

Próbáld ki

Az utolsó jegyedet, a 2-est kijavítottad 4-esre:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> jegyek = {5, 3, 4, 5, 2};

    std::cout << "Elotte: " << jegyek[4] << std::endl;
    jegyek[4] = 4;
    std::cout << "Utana: " << jegyek[4] << std::endl;
    return 0;
}
```

Tippelj

Mit ír ki ez a részlet? Kövesd sorról sorra papíron, és csak utána futtasd.

```
std::vector<int> v = {10, 20, 30};

v[0] = v[1] + v[2];
v[1] = v[1] * 2;
v[2]++;

std::cout << v[0] << " " << v[1] << " " << v[2] << std::endl;
```

Feladatok

A feladatokhoz használd újra a `lepesek` vektort: 8200, 10450, 6300, 12000.

3.1. feladat ★☆☆ - Javítás

Kiderült, hogy a második napon valójában 11000 lépést tettél meg. Javítsd ki a vektorban, és írd ki a javított értéket. Vigyázz: mi a második nap indexe?

3.2. feladat ★☆☆ - Esti séta

Az első napon este még sétáltál 500 lépést. Növeld meg az első elemet 500-zal. Ne a végeredményt írd be, a program számolja ki.

3.3. feladat ★☆☆ - Beolvasott adat

A negyedik nap adatát olvasd be a billentyűzetről. Egy vektorelembe ugyanúgy lehet beolvasni, mint egy változóba:

```
std::cin >> lepesek[3];
```

Írd ki utána mind a négy elemet, hogy lásd a változást.

3.4. feladat ★☆☆ - Csere

Cseréld meg az első és az utolsó elemet. Először próbáld ki ezt a két sort, és írasd ki az eredményt:

```
lepesek[0] = lepesek[3];  
lepesek[3] = lepesek[0];
```

Mi romlott el, és miért? Javítsd ki úgy, hogy a csere tényleg működjön. **Tipp:** kell egy harmadik, segédváltozó.

4. A vektor mérete és bejárása

A vektor igazi ereje az, hogy egy `for` ciklus végig tud menni az összes elemén, akárhány is van belőle.

Próbáld ki - hány elem van?

A vektor tudja magáról, hány eleme van. Ezt a `size()` árulja el:

```
std::vector<int> jegyek = {5, 3, 4, 5, 2};  
  
std::cout << "Jegyek száma: " << jegyek.size() << std::endl;
```

Adj hozzá a kapcsos zárójelbe még két jegyet, és futtasd újra.

Tippelj

Mit ír ki ez a sor? És miért pont azt?

```
std::cout << jegyek[jegyek.size() - 1] << std::endl;
```

Próbáld ki - az index lehet változó is

Eddig mindig számot írtál a szögletes zárójelbe. Változót is lehet:

```
int i = 2;  
std::cout << jegyek[i] << std::endl;
```

Írd át az `i` értékét 0-ra, majd 3-ra. Mindig az `i`-edik elemet kapod.

Állj meg itt egy percre, és gondolkodj: melyik utasítás tud egy változót 0-tól egyesével növelni?

Próbáld ki - a bejárás

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> jegyek = {5, 3, 4, 5, 2};

    for (int i = 0; i < jegyek.size(); i++) {
        std::cout << jegyek[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

A ciklusban az `i` sorban felveszi a 0, 1, 2, 3, 4 értékeket. Így a `jegyek[i]` sorban az összes elemet jelenti.

Tipp: Ha a fordító figyelmeztet (*warning: comparison of integer expressions of different signedness*), ne ijedj meg. Ez nem hiba, a program rendesen lefut.

Kísérletek

- Adj a vektorhoz még három jegyet. Hozzá kellett nyúlnod a ciklushoz? Emlékezz vissza az 1.2. feladatra.
- Írd át a feltételben a `<` jelet `<=` jelre. Mi történik? Miért? (Gondolj a 2. fejezet kísérletére.)
- Indítsd az `i`-t 1-ről 0 helyett. Melyik elem marad ki?

Feladatok

Használd ezt a vektort, most már egy egész hét adataival:

```
std::vector<int> lepesek = {8200, 10450, 6300, 12000, 9100, 15300, 4800};
```

4.1. feladat ★☆☆ - Egész hét

Írd ki az összes elemet, mindegyiket új sorba.

4.2. feladat ★☆☆ - Sorszámozva

Írd ki a nap sorszámát is, 1-től számozva:

```
1. nap: 8200
2. nap: 10450
...
```

Tipp: az index 0-tól indul, a sorszám 1-től. Mennyi a különbség?

4.3. feladat ★☆☆ - Visszafelé

Írd ki az elemeket visszafelé, az utolsótól az elsőig. Három kérdésre kell válaszolnod: honnan indul az `i`, meddig megy, és nő vagy csökken?

4.4. feladat ★☆☆ - Minden második

Írd ki csak minden második napot: az első, a harmadikat, az ötödiket és a hetediket. Kétféle megoldás is van, keresd meg mindkettőt.

4.5. feladat ★☆☆ - Tízezres napok

Írd ki csak azokat a napokat, amikor legalább 10000 lépést tettél meg. **Tipp:** a ciklus belsejébe `if` kerül.

4.6. feladat ★★☆☆ - Bónusz lépések

Mindegyik napra jóváírnak 1000 bónusz lépést. Egy ciklussal növel meg minden elem értékét 1000-rel. Utána egy másik ciklussal írd ki az új értékeket.

5. Bővítés és beolvasás

A vektor futás közben is nőhet: a `push_back` új elemet tesz a végére. Így azt is el tudod tárolni, amit a felhasználó gépel be.

Tippelj

Ebben a részletben három kiírás van. Mindhárom elé írd oda a tipped, és csak utána futtasd.

```
std::vector<int> szamok;
std::cout << szamok.size() << std::endl;

szamok.push_back(7);
szamok.push_back(3);
std::cout << szamok.size() << std::endl;

std::cout << szamok[0] << " " << szamok[1] << std::endl;
```

Amit megfigyelhettél:

- Ha nem adsz meg kezdő elemeket, a vektor üres, a mérete 0.
- A `push_back(x)` az `x` értéket a vektor **végére** teszi, és a méret eggyel nő.

Próbáld ki - beolvasás a billentyűzetről

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> szamok;

    int db;
    std::cout << "Hany szamot adsz meg? ";
    std::cin >> db;

    for (int i = 0; i < db; i++) {
        int x;
        std::cin >> x;
        szamok.push_back(x);
    }

    std::cout << "Ezeket adtad meg: ";
    for (int i = 0; i < szamok.size(); i++) {
        std::cout << szamok[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

A program két ciklusból áll. Az első beolvas és eltárol, a második kiír. A két ciklus között az adatok a vektorban várnak.

Futtasd többször, más-más darabszámmal. Próbáld ki 0-val is.

Kísérlet - kerek zárójel vagy kapcsos?

Írasd ki ciklussal ennek a vektornak az elemeit:

```
std::vector<int> nullak(5);
```

Mit látsz? Most hasonlítsd össze ezt a kettőt: írasd ki mindkettő méretét és elemeit.

```
std::vector<int> a(3);  
std::vector<int> b = {3};
```

Fogalmazd meg a füzetedben egy mondatban, mi a különbség a kettő között.

Feladatok

5.1. feladat ★☆☆ - Három szám

Hozz létre egy üres vektort. Tedd bele `push_back`-kel az 1, 2, 3 számokat, majd írd ki ciklussal.

5.2. feladat ★☆☆ - Egytől tízig

Töltsd fel a vektort 1-től 10-ig a számokkal. Most már ne tíz `push_back` sort írh, hanem egy ciklust.

5.3. feladat ★☆☆ - Négyzetszámok

Töltsd fel a vektort az első tíz négyzetszámmal: 1, 4, 9, 16, ... Írd ki az elemeket.

5.4. feladat ★☆☆ - Fordított hőmérsékletek

Kérj be öt hőmérsékletet, tárold el őket, majd írd ki fordított sorrendben. (A 4.3. feladat segít.)

5.5. feladat ★☆☆ - Nulláig

Kérj be számokat addig, amíg a felhasználó 0-t nem ír be. A 0 már ne kerüljön a vektorba. A végén írd ki, hány számot adott meg, és sorold fel őket. **Tipp:** itt nem tudod előre a darabszámot, ezért `while` ciklus kell.

5.6. feladat ★☆☆ - Kell-e tárolni?

Gondold végig, és írd le egy mondatban: az 5.4. feladatot meg lehetett volna oldani vektor nélkül, egyetlen `int` változóval?

6. Összegzés, átlag, megszámlálás

Ha egy ciklus végigmegy a vektoron, közben gyűjtögethet is: összeadhatja az elemeket, vagy megszámlálhatja, hány felel meg egy feltételnek.

Próbáld ki - összegzés

```
std::vector<int> jegyek = {5, 3, 4, 5, 2};  
  
int osszeg = 0;  
for (int i = 0; i < jegyek.size(); i++) {  
    osszeg = osszeg + jegyek[i];  
}  
  
std::cout << "Összeg: " << osszeg << std::endl;
```

Az `osszeg` olyan, mint egy persely: üresen indul, és minden körben beledobjuk a következő elemet.

Kövesd végig papíron, mi történik. Az első két sort kitöltöttem, a többi te írd be:

i értéke	jegyek[i]	összeg a kör végén
0	5	5
1	3	8
2		
3		
4		

Egyezik az utolsó sor azzal, amit a program kiírt?

Kísérlet

Tedd át az `int osszeg = 0;` sort a ciklus belsejébe, a nyitó kapcsos zárójel után. A kiírás maradjon a cikluson kívül. Lefordul a program? Ha nem, olvasd el a hibaüzenetet, és próbáld megmagyarázni.

Tippelj - átlag

Az átlag az összeg osztva a darabszámmal. A darabszámot ismered: `jegyek.size()`. Írd az összegző program végére ezt a sort:

```
std::cout << "Átlag: " << osszeg / jegyek.size() << std::endl;
```

Számold ki fejben, mennyi 19 osztva 5-tel. Aztán futtasd. Ugyanazt kaptad?

Két egész szám osztásakor a C++ levágja a tizedeseket. A megoldás: az összeg legyen `double` típusú.

```
double osszeg = 0;
```

Írd át, és futtasd újra.

Figyelem! Ha átlagot számolsz, az összeget mindig `double` változóban gyűjtsd. Két egész szám osztása C++-ban egész eredményt ad.

Próbáld ki - megszámolás

Hány ötösöd van?

```
std::vector<int> jegyek = {5, 3, 4, 5, 2};

int db = 0;
for (int i = 0; i < jegyek.size(); i++) {
    if (jegyek[i] == 5) {
        db++;
    }
}

std::cout << "Ötosok száma: " << db << std::endl;
```

Hasonlítsd össze az összegzéssel. Mi egyezik a két programban, és mi a két különbség?

Feladatok

Az első három feladathoz a heti lépésszámokat használd:

```
std::vector<int> lepesek = {8200, 10450, 6300, 12000, 9100, 15300, 4800};
```

6.1. feladat ★☆☆ - Heti összeg

Hány lépést tettél meg a héten összesen?

6.2. feladat ★☆☆ - Napi átlag

Mennyi a napi átlag? Tizedesjegyekkel együtt írd ki.

6.3. feladat ★☆☆ - Hány tízezres nap?

Hány napon érted el a 10000 lépést?

A következő kettőhöz egy téli hét hőmérsékletei kellenek:

```
std::vector<int> hom = {-3, 2, 0, -5, 4, 1, -1};
```

6.4. feladat ★☆☆ - Fagyos napok

Hány napon volt fagy, vagyis 0 foknál hidegebb?

6.5. feladat ★☆☆ - Fagypont felett

Mennyi a fagypont feletti értékek összege? Itt az összegzést és a feltételt kell összeraknod.

6.6. feladat ★☆☆ - Átlag feletti jegyek

Kérj be hat jegyet a billentyűzetről (5. fejezet). Írd ki az átlagukat, majd azt, hogy hány jegy jobb az átlagnál.

Gondold végig: hányszor kellett végigmenned az adatokon? Meg tudtad volna oldani úgy, hogy a jegyeket nem tárolod el?

7. Legnagyobb és legkisebb elem

A legnagyobb elemet úgy találod meg, hogy megjegyzed az eddigi legnagyobbat, és minden új elemnél megnézed, nagyobb-e nála.

Gondold végig gép nélkül

Valaki egyesével mutat neked számkártyákat, és mindegyiket csak egy pillanatra látod. A végén meg kell mondanod a legnagyobbat. Mit csinálsz közben fejben? Írd le két mondatban, mielőtt továbbolvasol.

Próbáld ki

```
std::vector<int> pontok = {42, 67, 55, 91, 38};

int legnagyobb = pontok[0];
for (int i = 1; i < pontok.size(); i++) {
    if (pontok[i] > legnagyobb) {
        legnagyobb = pontok[i];
    }
}

std::cout << "Legnagyobb: " << legnagyobb << std::endl;
```

Kövesd végig papíron: melyik körökben változik a **legnagyobb** értéke, és melyikekben nem?

Miért indulhat az **i** most 1-ről, és nem 0-ról?

Kísérlet - miért nem nulláról indulunk?

Kézenfekvőnek tűnik így kezdeni: `int legnagyobb = 0;`. Írd át a programot erre, és próbáld ki ezzel a vektorral:

```
std::vector<int> pontok = {-5, -2, -9};
```

Mit ír ki? Miért rossz ez az eredmény?

Figyelem! A legnagyobb elem keresését ne 0-ról indítsd! Mindig a vektor első eleméből indulj ki.

Feladatok

7.1. feladat ★☆☆ - Legkisebb

Írd át a mintaprogramot úgy, hogy a legkisebb pontszámot keresse meg. Hány helyen kellett érdemben változtatnod?

7.2. feladat ★☆☆ - Terjedelem

Írd ki egy programban a legnagyobb és a legkisebb pontszámot, valamint a kettő különbségét.

7.3. feladat ★☆☆ - Hol a legnagyobb?

Hányadik indexen van a legnagyobb elem? **Tipp:** most ne az értéket jegyezd meg, hanem a helyét.

```
int maxIndex = 0;
```

A ciklusban a `pontok[i]`-t a `pontok[maxIndex]`-szel hasonlítsd össze.

7.4. feladat ★☆☆ - Leghidegebb nap

Használd a téli hét hőmérsékleteit: -3, 2, 0, -5, 4, 1, -1. Írd ki, hányadik napon volt a leghidegebb (1-től számozva), és hány fok volt aznap.

7.5. feladat ★☆☆ - Két egyforma

Futtasd a 7.3. megoldásodat ezzel a vektorral: 3, 9, 4, 9, 1. Két egyforma legnagyobb van benne. Melyiknek az indexét kaptad? Mi történik, ha a `>` jelet `>=` jelre cseréled? Magyarázd meg.

8. Keresés: van-e benne, és hol?

Azt, hogy egy érték szerepel-e a vektorban, egy `bool` változó jegyzi meg: hamisról indul, és igazra vált, ha megtaláltuk.

Próbáld ki

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> szamok = {4, 8, 15, 16, 23, 42};

    int keresett;
    std::cout << "Melyik szamot keresed? ";
    std::cin >> keresett;

    bool van = false;
    for (int i = 0; i < szamok.size(); i++) {
        if (szamok[i] == keresett) {
            van = true;
        }
    }

    if (van) {
        std::cout << "Benne van." << std::endl;
    } else {
        std::cout << "Nincs benne." << std::endl;
    }
    return 0;
}
```

Futtasd háromszor: keresd a 15-öt, a 42-t és a 7-et.

Figyeld meg a szerkezetet. A ciklus csak keres és megjegyez. A válasz kiírása a ciklus **után** történik, amikor már minden elemet láttunk.

Kísérlet - a leggyakoribb hiba

Sokan így írják meg elsőre. Próbáld ki, és keresd vele a 15-öt:

```
for (int i = 0; i < szamok.size(); i++) {
    if (szamok[i] == keresett) {
        std::cout << "Benne van." << std::endl;
    } else {
        std::cout << "Nincs benne." << std::endl;
    }
}
```

Mit ír ki? Miért? Fogalmazd meg egy mondatban, mikor mondhatjuk ki biztosan, hogy egy szám nincs a vektorban.

Feladatok

8.1. feladat ★☆☆ - Hol van?

Egészítsd ki a mintaprogramot: ha megvan a keresett szám, azt is írja ki, hányadik indexen találta.

Tipp: vegyél fel egy `int hely = -1;` változót. Miért jó kezdőérték a -1?

8.2. feladat ★☆☆ - Van-e egyes?

A jegyeid: 5, 3, 4, 5, 2, 4, 3. Van köztük egyes? Írj rá programot. Aztán írd át egy jegyet 1-esre, és ellenőrizd, hogy a program észreveszi.

8.3. feladat ★★☆☆ - Mindegyik?

Igaz-e, hogy **minden** jegyed legalább hármas? **Tipp:** fordítva gondolkodj, mint eddig. Induljon a változó igazról (`bool mind = true;`), és akkor váltson hamisra, ha találsz egy kivételt.

8.4. feladat ★★☆☆ - A hatodik szám

Kérj be öt számot egy vektorba, majd kérj be egy hatodikat. Írd ki, hogy a hatodik szerepelt-e az első öt között.

8.5. feladat ★★☆☆ - Megállás az első találatnál

A mintaprogram akkor is végigmegy az egész vektoron, ha már az első elem a keresett. Ez a `while` ciklus megáll az első találatnál:

```
int i = 0;
while (i < szamok.size() && szamok[i] != keresett) {
    i++;
}
```

Futtasd, és írasd ki utána az `i` értékét, többféle keresett számmal. Mit jelent, ha az `i` kisebb, mint a vektor mérete? És ha éppen egyenlő vele? Fejezd be a programot úgy, hogy kiírja a választ.

9. Kihívások

Ezekhez a feladatokhoz nincs új tudnivaló: mindent ismersz hozzá, csak össze kell raknod. Nem kell sorban haladnod, válaszd azt, amelyik megtetszik.

9.1. feladat ★★☆☆ - Csillagdiagram

Adott a vektor: 3, 5, 1, 4. Rajzolj minden elemhez egy sort, amelyben annyi csillag van, amennyi az elem értéke:

```
***
*****
*
*****
```

Tipp: ciklus a ciklusban, ahogy az alakzatok rajzolásánál csináltad.

9.2. feladat ★★☆☆ - Szétválogatás

Adott a vektor: 12, 7, 4, 9, 20, 15, 8. Hozz létre két üres vektort `parosak` és `paratlanok` néven. Válogasd szét beléjük a számokat, majd írd ki mindkettőt.

9.3. feladat ★★☆☆ - Fordított másolat

Készíts egy új vektort, amelyben az eredeti elemei fordított sorrendben szerepelnek. Az eredeti vektor maradjon változatlan. Írd ki mindkettőt.

9.4. feladat ★★☆☆ - Növekvő-e?

Döntsd el, hogy a vektor elemei növekvő sorrendben vannak-e. Ehhez a szomszédokat kell összehasonlítnod: a `v[i]` elemet a `v[i + 1]` elemmel. Vigyázz: meddig mehet az `i`, hogy a `v[i + 1]` még létezzen? Próbáld ki mindkét vektorral:

```
std::vector<int> a = {1, 3, 7, 10};
std::vector<int> b = {1, 5, 4, 8};
```

9.5. feladat ★★☆☆ - Legnagyobb melegedés

A téli hét hőmérsékletei: -3, 2, 0, -5, 4, 1, -1. Melyik két egymást követő nap között nőtt a legtöbbet a hőmérséklet, és hány fokkal? Előbb számold ki fejben, hogy ellenőrizni tudd a programodat.

9.6. feladat ★★★ - Dolgozatstatisztika

Kérd be egy osztály dolgozatainak pontszámait (0 és 100 között). Előbb azt kérdezd meg, hány dolgozat van. Írd ki:

- az átlagpontszámot,
- a legjobb és a leggyengébb eredményt,
- hány dolgozat lett jobb az átlagnál,
- volt-e 100 pontos dolgozat.

Ez a feladat szinte az egész jegyzetet összefoglalja. Építsd lépésenként: egy rész megírása után futtasd, és csak utána írd a következőt.

9.7. feladat ★★★ - Megfordítás helyben

Fordítsd meg a vektor elemeinek sorrendjét új vektor nélkül. **Tipp:** a 3.4. feladat cseréjét használd. Cseréld meg az elsőt az utolsóval, a másodikat az utolsó előttivel, és így tovább.

Először menj végig a cserékkel az egész vektoron, és nézd meg az eredményt. Mi történt? Meddig kell tehát mennie a ciklusnak?

Összefoglaló táblázat (puska)

Ennyi kell ahhoz, hogy vektorral dolgozz. Minden programba kell az `#include <vector>` sor.

Művelet	Jelentés
<code>std::vector<int> v = {5, 3, 4};</code>	Vektor kezdő elemekkel
<code>std::vector<int> v;</code>	Üres vektor
<code>std::vector<int> v(5);</code>	Öt darab nulla
<code>std::vector<double> v;</code>	Tizedes törtek vektora
<code>v[i]</code>	Egy elem elérése
<code>v[i] = 10;</code>	Egy elem módosítása
<code>v.size()</code>	Elemek száma
<code>v[0]</code>	Első elem
<code>v[v.size() - 1]</code>	Utolsó elem
<code>v.push_back(x);</code>	Új elem a végére
<code>for (int i = 0; i < v.size(); i++)</code>	Bejárás

Figyelem! Két dolog, amit soha ne felejts el: az indexek 0-tól `v.size() - 1`-ig tartanak, és a C++ nem szól, ha ezen kívülre nyúlsz.